

Reproducing “*No cosmological constraints on dark photon dark matter from resonant conversion*” (arXiv:2510.13956) with Entity Implementation on Lonestar6 and comparison with the published runs

September 15, 2026

Abstract

This note documents a reproduction of the particle-in-cell (PIC) results of Hook, Huang & Shalaby (arXiv:2510.13956v1) using the ENTITY PIC framework in place of the authors’ SHARP code.

Part [I](#) is an operating manual: the repository layout, a quickstart, a reference for the input deck, job scripts and queue selection, the output format and analysis pipeline, a procedure for sizing a new run, and a consolidated list of the rules and traps that govern all of it. It is reference-ordered and is the only part needed in order to run something. Part [II](#) covers the implementation: a dedicated problem generator, a deterministic (“quiet”) particle loader written for this work, the four build configurations required on TACC Lonestar6, and a measured performance characterisation across CPU, single-GPU and multi-GPU execution. Part [III](#) documents every production run, the exact settings used, and a point-by-point comparison with the parameters and results stated in the paper. Both resonance cases of the paper’s Fig. 2 are reproduced; the Landau–Zener family is in progress. All performance and physics numbers quoted are measured on Lonestar6 unless explicitly labelled as an estimate.

Contents

1	Scope and conventions	2
I	Working guide	4
2	Repository map	4
3	Quickstart	4
4	The input deck	5
4.1	The [setup] block — the DPDM interface	5
4.2	Settings outside [setup] that are not optional	5
4.3	A complete deck	6
5	Job scripts and the run loop	7
5.1	Template	7
5.2	Choosing a queue	8
5.3	Checkpoints and restarting	8

6	Output and analysis	8
6.1	What a run writes	8
6.2	Deriving the physical quantities	9
6.3	Reading the field dumps	9
7	Sizing a new run	9
8	Rules and traps, consolidated	10
II	Implementation	12
9	Problem generator	12
10	Deterministic (“quiet”) particle loading	12
10.1	What a quiet start must actually kill	12
10.2	Construction	13
10.3	Validation	13
10.4	What the quiet start does <i>not</i> buy	14
11	Building on Lonestar6	14
11.1	Environment and prerequisites	14
11.2	Configure	14
11.3	Compile	15
12	Measured performance	16
12.1	Throughput	16
12.2	Per-step cost doubles after saturation	16
III	Runs, settings, and comparison with the paper	17
13	Parameters taken from the paper source	17
14	Preparatory and calibration runs	17
15	The two resonance cases (paper Fig. 2)	18
16	Landau–Zener runs	18
17	Superseded results	19
18	Status and remaining work	19

1 Scope and conventions

The paper studies the resonant conversion of dark-photon dark matter (DPDM) into a plasma excitation in the early-universe electron–ion plasma, and shows that *nonlinear* plasma dynamics saturate the conversion far below the linear-theory expectation, weakening the resulting cosmological limits. The evidence is a set of 1D–1V electrostatic PIC simulations.

Stripped of the cosmology, the numerical problem is small and sharply defined:

- 1D electrostatic, periodic box, $N_x = 1000$, $L = 40 c/\omega_p$, $\Delta x = 0.04 c/\omega_p \simeq \lambda_D$;
- electrons and ions with the *real* mass ratio $m_i/m_e = 1836$, both evolved;

- $k_B T_e = k_B T_i = 10^{-3} m_e c^2$;
- the dark photon enters as a *spatially uniform* ($k = 0$), time-oscillating external electric field $E_{\text{ext}}(t) = A_0 \cos(\omega t)$ acting on the particle Lorentz force *only* — it is never a source term in Maxwell’s equations. All feedback is through the deposited current.

Code units. Throughout, $\omega_{pe} = 1$, $c = 1$, $n_e = n_0$, $m_e = 1$. Energies are in $n_e m_e c^2$, times in ω_p^{-1} , wavenumbers in ω_p/c . The drive amplitude in code units equals the paper’s: $A_0 = v_q^D/c = (v_q^D/v_{\text{th}}^e)\sqrt{10^{-3}}$.

Measured versus estimated. Every performance and physics number here is measured on Lonestar6 unless explicitly labelled an estimate. Section 17 lists the few figures from earlier drafts that are superseded, so that they are recognisable if they turn up in older scripts or notes.

Part I

Working guide

This part is reference-ordered and self-contained: it is what to read in order to run something. Part II explains why the code is built the way it is, and Part III records what the production runs produced. Neither is needed to operate the code.

2 Repository map

Everything specific to this work lives in two directories of the ENTITY tree.

Path (under repository root)	Contents
<i>pgens/dpdm/ — problem generator, decks, job scripts</i>	
pgen.hpp	The problem generator: external drive, quiet loader, custom diagnostic. The only C++ in this work (§9).
resonance.toml	Reference deck, fixed-frequency family. Start here.
landau.zener.toml	Reference deck, swept-frequency family.
run_paper_fast.sh	Production: $v_q^D/v_{th}^e = 3 \times 10^{-2}$, $N_{pc} = 2 \times 10^2$, 1 CPU node.
run_paper_slow.sh	Production: $v_q^D/v_{th}^e = 1 \times 10^{-3}$, $N_{pc} = 2 \times 10^5$, 1 A100.
run_paper_slow_resume.sh	The same run restarted from checkpoint (§5.3).
run_lz1p0.sh, run_lz0p1.sh	Landau–Zener production at $v_q^D/v_{th}^e = 1.0$ and 0.1 .
run_lz_heating.sh	Undriven numerical-heating gate at Landau–Zener duration.
validate_quiet.sh, check_quiet.py	Quiet-start validation gates (§10.3).
validate_mpi.sh	Confirms results are independent of MPI domain count.
bench_slow.sh, bench_gpu.sh, bench_multigpu.sh	Throughput benchmarks: CPU/MPI, 1 GPU, 3 GPU.
diag_slow.sh, diag_growth.sh, affinity_test.sh	Diagnostics for performance and thread-placement questions.
<i>pgens/dpdm/paper/ — analysis pipeline for the paper-faithful runs</i>	
extract_paper.py	Reduces fast , slow , lz1p0 , heat and the quiet/random pair to .npz (§6).
LOTS.md	The six plots those runs support, and what each claims.
<i>pgens/dpdm/run34/ — the resonance-ladder survey and its analysis pipeline</i>	
run34.sh	Eight-run ladder in v_q^D/v_{th}^e plus two convergence runs.
extract_run34.py	Reduces ADIOS2 output to .npz (§6).
LOTS.md	Which four plots to make and what each one claims.
run34_plots.ipynb	The plots, from the .npz files.
<i>dpdm_doc/ — this document</i>	
main.tex, Makefile	Source and build (make → main.pdf).

Build directories (**build-dpdm***) are deliberately not version controlled; see §11. Run output goes to **\$SCRATCH**, which is **purged on a rolling basis** — copy anything you intend to keep to **\$WORK**.

3 Quickstart

End to end, on a login node unless stated otherwise:

1. **Build once.** Follow §11. For a first run the CPU OpenMP configuration (**build-dpdm**) is enough; the CUDA build (**build-dpdm-cuda**) is what production uses.
2. **Verify the build** — configure options fail silently here:

```
grep -E "^Kokkos_ENABLE_(OPENMP|SERIAL|CUDA):BOOL" build-*/CMakeCache.txt
```

3. **Copy a job script** from `pgens/dpdm/` that matches the shape of what you want — they are self-contained and embed their own deck — and edit the deck inside it (§4).
4. **Size the run:** particle count, GPU count and walltime follow a fixed procedure (§7). The GPU count in particular cannot be changed after the first submission if the run will ever restart.
5. **Submit:** `sbatch run_my_case.sh`. Jobs under 2h belong in development (§5.2).
6. **Analyse:** the stats CSV answers most questions directly; field and particle dumps go through `extract_run34.py` (§6).

4 The input deck

4.1 The `[setup]` block — the DPDM interface

These nine keys are the whole problem-specific interface; everything else in the deck is stock ENTITY. Defaults are those in `pgen.hpp`.

Key	Default	Meaning
<code>drive</code>	<code>"resonance"</code>	<code>"resonance"</code> (fixed ω) or <code>"landau.zener"</code> (swept).
<code>loading</code>	<code>"random"</code>	<code>"quiet"</code> selects the deterministic lattice. Set this explicitly — the default is the stock random injector (§10).
<code>one_v</code>	<code>true</code>	Sets $u_{x2} = u_{x3} = 0$, matching SHARP and the paper's 1V thermal speed. Leave on for paper comparison.
<code>A0</code>	<code>0.0</code>	Drive amplitude in code units, $A_0 = v_q^D/c = (v_q^D/v_{th}^e)\sqrt{10^{-3}}$.
<code>omega</code>	<code>1.0</code>	Drive frequency, <code>drive="resonance"</code> only.
<code>omega0</code>	<code>0.0</code>	Initial frequency, <code>drive="landau.zener"</code> only.
<code>domega_dt</code>	<code>0.0</code>	Sweep rate, <code>drive="landau.zener"</code> only.
<code>densities</code>	<code>[2.0]</code>	One entry, per species <i>pair</i> . <code>[2.0]</code> with <code>skindpth0 = 1</code> gives $n_e = n_0$, hence $\omega_{pe} = 1$.
<code>temperatures</code>	<code>—</code>	One entry per species, in $m_0 c^2$. The injector divides by species mass, so <code>[1.0e-3, 1.0e-3]</code> is $10^{-3} m_e c^2$ for both.

The two drive families are mutually exclusive in their parameters: `omega` is ignored when sweeping, and `omega0/domega_dt` are ignored when not. An unrecognised `drive` or `loading` aborts at startup with a readable message rather than running the wrong physics.

4.2 Settings outside `[setup]` that are not optional

- `particles.ppc0` must be written as a **float**. `ppc0 = 2000` aborts with `toml::value::as_floating(): bad_cast`; write `2000.0`.
- `maxnpart` per species must exceed `ppc0 × densities[0]/2 × Nx`, with headroom.
- A `[diagnostics]` block is required in practice. Without it, `<name>.log` is a kernel trace at $\sim 4\text{ kB per step}$ and `<name>.out` is a progress bar redrawn every step, both written synchronously to the parallel filesystem. Use `log_level = "WARNING"`, `interval = 1000`, `colored_stdout = false`.
- `[output.spectra]` defaults to *on*; set `enable = false`.
- `[output.stats]` `interval_time` must be ≥ 0.5 , and preferably ≥ 1.0 . Per-species `T00.1/T00.2` force a moment reduction over every particle; at 0.1 this doubles wall-clock.

- Stats quantity names are literal: "E²", "T00_1", "T00_2", "J.E". "JdotE" aborts. "T00" alone gives the species sum.
- `algorithms.current_filters = 0`. The saturation physics lives at high k ; filtering removes exactly the modes of interest.

4.3 A complete deck

The fast resonance case in full. Every other run in this work is this file with `ppc0`, `A0`, `runtime` and the `[setup]` drive keys changed.

```
[simulation]
  name = "fast"
  engine = "srpic"
  runtime = 10000.0          # omega_p t
[grid]
  resolution = [1000]        # Nx
  extent = [[0.0, 40.0]]     # L = 40 c/omega_p, so dx = 0.04 ~ lambda_D
[grid.metric]
  metric = "minkowski"
[grid.boundaries]
  fields = [["PERIODIC"]]
  particles = [["PERIODIC"]]
[scales]
  skinddepth0 = 1.0          # with densities=[2.0] this sets omega_pe = 1
  larmor0 = 1.0              # unused: no magnetic field
[algorithms]
  current_filters = 0         # KEEP AT 0 -- see above
[algorithms.timestep]
  CFL = 0.5                  # dt = 0.02 / omega_p
[particles]
  ppc0 = 200.0               # N_pc per cell per species; Np = N_pc*Nx = 2e5
  [[particles.species]]
    label = "e-"
    mass = 1.0
    charge = -1.0
    maxnpart = 2.5e5
    pusher = "Vay"
  [[particles.species]]
    label = "i+"
    mass = 1836.0             # real mass ratio; ions are evolved
    charge = 1.0
    maxnpart = 2.5e5
    pusher = "Vay"
[setup]
  densities = [2.0]
  temperatures = [1.0e-3, 1.0e-3]
  drive = "resonance"
  loading = "quiet"          # NOT the default -- must be stated
  one_v = true
  A0 = 9.48683e-4            # v_q^D/vth_e = 3e-2
  omega = 1.0
[diagnostics]
  interval = 1000
  log_level = "WARNING"
  colored_stdout = false
[output]
  interval_time = 10.0        # field/particle dumps; 1000 of them here
[output.fields]
  quantities = ["E", "N_1", "N_2", "Rho", "J", "T00_1", "T00_2"]
[output.particles]
  species = [1, 2]
  stride = 20
[output.stats]
```

```

    enable = true
    interval_time = 0.5
    quantities = ["E^2", "T00_1", "T00_2", "J.E"]
    custom = ["e_ext"]          # the drive itself, for phase checks
[output.spectra]
    enable = false
[checkpoint]
    interval_time = 2500.0
    keep = 2

```

For the swept family, replace the four drive keys:

```

drive = "landau_zener"
A0 = 3.16228e-3          # v_q^D/vth_e = 0.1
omega0 = 0.8             # start below resonance
domega_dt = 2.0e-5       # 0.1 omega_p over omega_p t = 5000

```

5 Job scripts and the run loop

5.1 Template

Job scripts in `pgens/dpdm/` are self-contained: they carry their own deck as a heredoc, so a script plus its log is a complete record of what was run. The skeleton:

```

#!/bin/bash
#SBATCH -J dpdm_fast
#SBATCH -o sbatch_logs/out/fast.o%j
#SBATCH -e sbatch_logs/err/fast.e%j
#SBATCH -p development          # see queue selection below
#SBATCH -N 1
#SBATCH -n 1
#SBATCH -t 01:30:00
#SBATCH -A PHY23028
#SBATCH --mail-type=all
#SBATCH --mail-user=<you>@utexas.edu

module load gcc/13.2.0          # REQUIRED at run time, not only at build time

export OMP_NUM_THREADS=112      # CPU builds only; omit for CUDA
export OMP_PROC_BIND=close
export OMP_PLACES=cores

ROOT=$SCRATCH/dpdm/paper/fast
EXE=<repo>/build-dpdm/src/entity.xc
mkdir -p $ROOT/fast            # pre-create <simulation.name>/ -- see below

cat > $ROOT/fast.toml <<EOF
... deck ...
EOF

cd $ROOT && $EXE -input fast.toml > fast.stdout 2>&1
echo "exit=? $(date -Is)"

```

Three things in that skeleton are load-bearing:

- `module load gcc/13.2.0` inside the job. The binary links GCC 13's `libstdc++`; the default environment is `gcc/9.4.0` and the executable dies with `GLIBCXX_3.4.31 not found` *before* reading its input.
- `mkdir -p $ROOT/<simulation.name>`. Under MPI every rank calls `mkdir` on that directory and the losers abort with `filesystem error: cannot create directory: File exists, exit 255`, within seconds. A job that "finished" in four seconds is this, not speed.

- `sbatch_logs/out/` and `sbatch_logs/err/` must exist before submission or the job is rejected.
- MPI runs replace the last line with `ibrun` and set `-n` to the rank count; single-GPU runs need no launcher and no `OMP_*` variables.

5.2 Choosing a queue

Partition	MinNode	MaxNode	MaxWall	MaxNodePU	MaxJobsPU
development	1	8	02:00:00	8	1
normal	1	64	2-00:00:00	64	20
gpu-a100	1	8	2-00:00:00	12	8
gpu-a100-dev	1	2	02:00:00	2	1
gpu-a100-small	1	1	2-00:00:00	3	3

- **Under 2 h and ≤ 8 nodes: use development.** Measured turnaround there was 9 s and 70 s for jobs that never started at all in normal. `MaxJobsPU = 1` only means short jobs run back-to-back, which still beats normal priority.
- **Single-GPU production: gpu-a100-small.** One A100 per node, billing weight 32, versus three per node at weight 128 on gpu-a100.
- **Prefer fewer nodes than the scaling curve suggests.** On a busy machine queue wait swamps compute: multi-node GPU jobs requesting 2 and 4 nodes sat unscheduled for 30 h and were cancelled, while single-node jobs started within minutes. A 12-GPU configuration with a 6 h runtime is worthless if it waits 30 h for contiguous nodes.

5.3 Checkpoints and restarting

Checkpoints are controlled by `[checkpoint] interval_time` with `keep` retained, and written to `<name>.ckpt/` as `step-NNNNNNNN.bp` plus a `meta-NNNNNNNN.toml`. A run resumes with

```
cd $ROOT && $EXE -input fast.toml -resume > fast.resume.stdout 2>&1
```

(aliases: `-continue`, `-restart`, `-checkpoint`). `ENTITY` selects the latest checkpoint automatically. Increase runtime in the deck if the original value has already been reached.

The domain count is locked by the checkpoint format. An N -domain checkpoint can only be resumed by N domains: a single-GPU checkpoint cannot be resumed by a 3-GPU binary. Decide the rank/GPU count before the *first* submission of any run that might need to restart — there is no way to change it later without starting over.

6 Output and analysis

6.1 What a run writes

Output lands in `<rundir>/<simulation.name>/:`

File	Contents
<code><name>_stats.csv</code>	Scalar time series, one row per <code>[output.stats] interval_time</code> . Answers most questions on its own.
<code>fields/*.bp</code>	ADIOS2 BP5 field dumps at <code>[output] interval_time</code> .
<code>particles/*.bp</code>	Particle dumps, decimated by <code>[output.particles] stride</code> .
<code><name>.log</code> , <code><name>.out</code> , <code><name>.info</code> , <code><name>.err</code>	Logs; see §4.2 before the first production run.
<code><name>.ckpt/</code>	Checkpoints (sibling of <code><name>/</code> , not inside it).

The stats CSV columns, in order, are `step`, `time`, `E1^2`, `E2^2`, `E3^2`, `T00_1`, `T00_2`, `J.E`, and any custom entries (here `e.ext`). In 1D electrostatic runs `E2^2` and `E3^2` are zero by construction.

6.2 Deriving the physical quantities

The conversions below are the ones the plots need, and two of them are easy to get wrong in a way that changes conclusion by a factor of two or three.

- $\epsilon_E = \frac{1}{2}\langle E_x^2 \rangle$ from `E1^2`.
- $\Delta KE_e = T00_1(t) - T00_1(0)$, and likewise for the ions. Taking the difference is what makes this safe under ENTITY's 3V convention: with no transverse forces the perpendicular energy is constant and cancels.
- **The linear-theory line is $\epsilon_E + \Delta KE_e = CA_0^2(\omega_p t)^2$ with $C = 1/8$, compared against the *sum*, never ϵ_E alone.** A resonantly driven Langmuir wave equipartitions, so the field holds half the wave energy on cycle average and swings between 3% and 85% of it instantaneously at $2\omega_p$.
- **The electron thermal reference is the 1V value** $\epsilon_{th} = \frac{1}{2}n_e v_{th,e}^2 = 5 \times 10^{-4}$, not `T00_1 - 1 =  $1.5 \times 10^{-3}$` , which is the 3V rest-frame value.
- `Rho` in the field output is *mass* density ($1 + 1836 = 1837$ in code units); the charge density is the separate `Charge` field.
- `CustomStat` receives `time` one step behind the `time` column of the same row (`engine.hpp` passes `time - dt` to the writer). This affects `e_ext` bookkeeping only; the pusher's centring is correct.

6.3 Reading the field dumps

Field and particle output is ADIOS2 BP5 and is read with `nt2py`:

```
pip install nt2py                # brings adios2 + xarray + dask
```

```
import nt2
data = nt2.Data("<run_dir>/<name>/<name>")    # note: name repeated
```

`pgens/dpdm/run34/extract_run34.py` is the worked pipeline: it reduces a full survey (~ 2.7 GB of `.bp`) to ~ 65 MB of `.npz` — `<tag>_stats.npz`, `<tag>_fields.npz`, `<tag>_temps.npz`, plus `summary.npz` and `meta.json` — so that plotting needs only `numpy`. Set `run34_dir` at the top of the file and run it once. This is the intended route for analysing on a laptop: reduce on the cluster, download the `.npz`.

`run34/LOTS.md` states which four plots matter and what each one claims: the energy budget (paper Fig. 2), the $E(k, t)$ and $n(k, t)$ spectra that show *why* it saturates, the N_p convergence gate, and the fast/slow regime transition. `run34_plots.ipynb` produces them from the `.npz`.

`pgens/dpdm/paper/extract_paper.py` is the same pipeline for the production runs, which `run34` cannot substitute for: `run34` is a random-start survey at a uniform $N_{pc} = 2000$ out to $\omega_p t = 3000$, whereas these are quiet-start, at the paper's own N_{pc} , and include the swept-frequency family. It covers `fast`, `slow`, `lz1p0`, `heat` and the `undriven_q/undriven_r` pair, adding (x, u_x) phase-space histograms to the products, and reduces ~ 1.9 GB of `.bp` to ~ 50 MB. Stage the runs without their checkpoint directories first — `slow` alone carries 75 GB of checkpoints beside 483 MB of science. `paper/LOTS.md` states the six plots.

7 Sizing a new run

Given a dark-photon amplitude v_q^D/v_{th}^e , in order:

1. **Amplitude.** $A_0 = (v_q^D/v_{th}^e)\sqrt{10^{-3}}$.
2. **Particle count.** Use the paper's shot-noise floor $\epsilon_{noise} = L^2/(12N_p N_x)$, in which N_p is the **total macroparticle count**, $N_p = N_{pc} N_x$ — not the per-cell count. So evaluated, it

agrees with measurement at a flat factor 3.1 across three decades of N_{pc} , and is the right tool for this job. In code units the measured floor is $\epsilon_{\text{noise}} = 2.06 \times 10^{-4}/N_{\text{pc}}$.

- *Resonant drive*: the signal grows as t^2 , so it need only outgrow the noise before saturation.
- *Swept drive*: the pre-resonance response is a *bounded* forced oscillation, $A_0/(1 - \omega^2/\omega_p^2) = 2.78A_0$ at $\omega = 0.8\omega_p$, so there is no time at which signal outgrows noise. Use the paper’s criterion $\bar{t}_{\text{noise}} = L/(A_0\sqrt{3N_p N_x/2}) \leq 0.02$, giving $N_{\text{pc}} = 2.7 \times 10^3$ at $v_q^D/v_{\text{th}}^e = 1.0$ and 2.7×10^5 at 0.1.

A quiet start does not reduce N_{pc} ; see §10.4.

3. **Memory, hence GPU count.** At $N_{\text{pc}} = 2.7 \times 10^5$ a run holds 5.3×10^8 particles and ≈ 31 GB — tight on one 40 GB A100, comfortable at ≈ 10.5 GB per GPU across three. Because the checkpoint domain count is locked (§5.3), settle this before submitting.
4. **Walltime.** Take the per-step cost from Table 2, then **double it**, or benchmark across the saturation transition. Per-step cost roughly doubles once the plasma becomes turbulent ($265 \rightarrow 516$ ms on one A100 at $N_{\text{pc}} = 2 \times 10^5$); a benchmark run entirely inside the laminar phase measures the wrong number. Steps = runtime/0.02.
5. **Queue.** §5.2. If the estimate exceeds the 48 h cap, plan the restart chain from the start.

8 Rules and traps, consolidated

Everything below is a standing rule. Each is stated once here; the evidence for it is in Parts II and III.

Building.

1. Configure and compile on a **login node**: FetchContent needs outbound network and compute nodes have none.
2. `module load gcc/13.2.0` at build time *and* in every job script.
3. Pass `-D Kokkos_ENABLE_OPENMP=ON -D Kokkos_ENABLE_SERIAL=OFF` explicitly. `CMAKE_IGNORE_PATH`, which is required to make the ADIOS2 fetch work, also blocks the repository’s Kokkos settings, and a freshly fetched Kokkos defaults to Serial only — a build that is externally indistinguishable and $\sim 112\times$ slower. Neither SLURM’s `-c` default nor `ibrun` binding causes this; do not go looking there.
4. Always verify the cache after configuring (§3, step 2).

Decks.

5. `ppc0` must be a TOML float.
6. `loading = "quiet"` must be stated; the default is "random".
7. Include `[diagnostics]`, disable `[output.spectra]`, keep `[output.stats] interval_time ≥ 1.0`, keep `current_filters = 0`.

Running.

8. Pre-create `<rundir>/<simulation.name>/` before an MPI run.
9. Fix the rank/GPU count before the first submission: checkpoints lock it.
10. Budget a factor two on per-step cost across saturation.
11. Jobs under 2 h go to `development`; prefer fewer nodes.
12. `blocking_timers = true` is the first move on any performance surprise. It prints a per-kernel breakdown in ns per particle-step and settles in seconds questions that speculation gets wrong.

Analysis.

13. Compare linear theory against $\epsilon_E + \Delta K E_e$, not ϵ_E .
14. Use the 1V thermal reference 5×10^{-4} .
15. N_p in the shot-noise formula is the total macroparticle count.
16. `Rho` is mass density; charge density is `Charge`.
17. $E_x(t=0) = 0$ proves nothing about the loading — ENTITY initialises E to zero identically and both loaders co-locate the species. The discriminator is the total density $n_e + n_i$.

Part II

Implementation

9 Problem generator

All problem-specific code lives in `pgens/dpdm/pgen.hpp`. It provides three things.

External field. An `ExternalFields` hook returning a uniform e_{x1} , applied to *both* species; the pusher scales by q/m per species, so the ions automatically receive $1/1836$ of the electron kick. Because the drive is uniform in space, the cosine is evaluated once per species per step on the host and only the scalar is handed to the kernel — not once per particle.

Two drive families. Selected by `setup.drive`:

$$\text{resonance: } \phi(t) = \omega t, \quad \omega = \omega_p \text{ fixed}, \quad (1)$$

$$\text{Landau-Zener: } \phi(t) = \omega_0 t + \frac{1}{2} \dot{\omega} t^2. \quad (2)$$

The swept phase is integrated *analytically*. Sampling $\omega(t)$ per step and multiplying by t accumulates a spurious chirp: the naive form deviates by up to $0.86 A_0$ over the sweep. The analytic form was verified against the closed expression to 2.8×10^{-14} (double precision, i.e. exact).

A custom diagnostic. `CustomStat` exposes `e_ext`, the instantaneous drive, so amplitude and phase can be checked against the plasma response in the same CSV. Note that `CustomStat` receives time one timestep behind the `time` column in the same row (`engine.hpp` passes `time - dt` to the writer); this is a diagnostic bookkeeping offset only — the pusher’s centring is correct, as the drive-normalisation measurement in §14 confirms.

10 Deterministic (“quiet”) particle loading

The paper initialises “all particles equally spaced, with ions and electrons colocated throughout the computational domain,” which ensures $\epsilon_{\text{noise}} = 0$ at $t = 0$. ENTITY’s stock injector places particles at *random* sub-cell positions and has no quiet-start option, so one was written for this work.

10.1 What a quiet start must actually kill

There are three independent noise channels, and the phrase “equally spaced” only obviously addresses one:

1. **Charge density.** $\rho(x, 0) \neq 0$ seeds $E(x, 0) \neq 0$. Killed by co-locating each electron with an ion. ENTITY’s *random* start already does this, so this channel was never the problem.
2. **Total density.** $n_{\text{tot}} = n_e + n_i$ fluctuates at $\sqrt{2/N_{\text{ppc}}}$ even with perfect co-location, because both species fluctuate together. Since $\omega_p \propto \sqrt{n}$, this is a spread in the local resonant frequency — the channel that physically matters.
3. **Velocity space.** Even on a perfect lattice, randomly sampled velocities produce current fluctuations that regenerate E within a plasma period and re-seed channel 2 dynamically.

A loader that fixes 1 and 2 but not 3 looks perfect at $t = 0$ and then relaxes to the random-start floor within a few ω_p^{-1} .

10.2 Construction

The essential requirement is that **each velocity class must itself be a uniform lattice over the box**. Then under free streaming a class merely translates, and a Δx -spaced lattice deposits *exactly* uniform density at any translation, because the shape function is a partition of unity at that spacing. Indexing by velocity class rather than by sub-cell slot:

```
c = p / ncell          // velocity class: exactly ncell particles each
i = p % ncell          // cell
phi = ((c*P0) mod ppc + 1/2)/ppc  // sub-cell offset: a function of c ALONE
x = xi_min + i + phi
u_e = sig1 * probit((c + 1/2)/ppc)
u_i = sig2 * probit((c*P2) mod ppc + 1/2)/ppc)
```

Four properties, each doing a specific job:

- **phi** depends only on **c**, so every velocity class is a perfect Δx lattice. *This is what makes the order survive streaming.*
- **P0**, **P2** coprime to **ppc** make the index maps bijections, so each cell carries the complete stratified quantile set exactly once (zero net drift, exact temperature per cell) while consecutive slots receive *non*-consecutive quantiles, so no cell holds a systematic $v(x)$ ramp.
- Ions reuse the electron positions, so the charge density vanishes bit-for-bit at $t = 0$; their velocities use a second stride so the species are not phase-space correlated.
- Nothing references the cell index or the domain offset, so the loading is independent of how MPI decomposes the domain.

Why not the obvious construction. Tying the velocity to the sub-cell *slot* with a per-cell rotation is the natural first design and it does not work. A given velocity then sits at a different offset in each cell, so the classes shear apart as soon as particles move: the measured result is $\delta n/n = 6.3 \times 10^{-3}$ after a *single* step, with a noise floor only $2\times$ below a random start. Getting $t = 0$ clean is easy; making the order survive streaming is the requirement.

Velocity normalisation. The stratified set $\{ \text{probit}((j + \frac{1}{2})/N_{\text{pc}}) \}$ has mean zero by symmetry but variance *below* unity, because midpoint quantiles under-sample the Gaussian tails: -1.97% at $N_{\text{pc}} = 64$, -0.64% at $N_{\text{pc}} = 200$, -0.07% at $N_{\text{pc}} = 2000$. A device-side reduction measures the realised variance and divides it out, so the loaded temperature is exact. Without this the plasma runs systematically cold at production particle counts — an invisible bias of exactly the kind a quiet start exists to remove. **probit** is Acklam’s rational approximation plus one Halley step against **erfc**, accurate to $\sim 10^{-16}$ over $q \in [10^{-8}, 1 - 10^{-8}]$.

10.3 Validation

Gate	Result	Reference
Lattice uniformity	$\delta n/n = 0.000 \times 10^0$ (exactly)	random: $1/\sqrt{N_{\text{pc}}} = 0.125$
Charge neutrality	$\max \rho_q = 0.000 \times 10^0$	—
Undriven noise floor	1.31×10^{-17}	random: 3.03×10^{-6}
No coherent seeded mode	peak/median = 10.6, power $\sim 10^{-38}$	—
Drive normalisation	$C = 0.12531$	theory $1/8 = 0.125$
8-domain MPI vs 1 domain	agreement to 1.5×10^{-12}	reduction ordering

Table 1: Quiet-start validation gates, $N_{\text{pc}} = 64$. Scripts: `validate-quiet.sh`, `check-quiet.py`, `validate-mpi.sh`.

Two traps worth recording. **ENTITY**’s **Rho** field is *mass* density ($1 + 1836 = 1837$); the charge density is the separate **Charge** field. And $E_x(t=0) = 0$ proves nothing, because **ENTITY** initialises

E to zero identically and *both* loadings co-locate the pair — the discriminator is the *total* density $n_e + n_i$.

10.4 What the quiet start does *not* buy

An undriven run at $N_{\text{pc}} = 2700$ over the full Landau–Zener duration ($\omega_p t = 2 \times 10^4, 10^6$ steps) shows the ordered state *relaxing*:

ϵ_E at $t = 0$	6.7×10^{-23}
ϵ_E plateau (reached by $\omega_p t \approx 5000$)	4.8×10^{-8}
random-start floor at the same N_{pc}	7.6×10^{-8}

The lattice starts at round-off and relaxes to $\sim 62\%$ of the thermal shot-noise floor. This is the paper’s own statement (“it takes approximately $t\omega_p \sim 3\text{--}4$ for the driven energy to be converted into de-coherent high- k noise”) seen from the undriven side. **Consequence: a quiet start does not permit a reduction in N_{pc} .** It buys a clean $t = 0$ and nothing more; particle counts must still be sized from the paper’s shot-noise criterion.

The same run bounds numerical heating over 10^6 steps with ordered loading:

$$\Delta(k_B T_e)/k_B T_e = -0.0162\%, \quad \Delta(k_B T_i)/k_B T_i = +0.0084\%.$$

An ordered lattice does not drift. This was the gate for the Landau–Zener runs, where a slow T_e creep would shift ω_p and masquerade as physics.

11 Building on Lonestar6

This section is the full recipe: the four build directories used in this work can be reconstructed from scratch by following it in order. The directories themselves are not version-controlled — they total ~ 1.2 GB of object files, fetched dependency sources and architecture-specific binaries, and are valid only on this machine.

11.1 Environment and prerequisites

All four builds are configured and compiled **on a login node**. ENTITY fetches Kokkos and ADIOS2 with CMake’s `FetchContent` at configure time, and LS6 compute nodes have no outbound network: the same commands on a compute node fail in the clone step.

```
module load gcc/13.2.0    # switches the hierarchy; brings impi/21.12 with it
module load cuda/12.8    # CUDA builds only -- LS6 defaults to cuda/11.4
```

`gcc/13.2.0` is not optional. The binaries link against GCC 13’s `libstdc++`; the default LS6 environment provides `gcc/9.4.0`, under which the executable dies at load time with `GLIBCXX.3.4.31 not found before` it reads its input file. The module must therefore also be loaded in every job script, not only at build time. Loading it switches the module hierarchy to `/opt/apps/gcc13.2/modulefiles` and pulls in `impi/21.12` (Intel oneAPI MPI 2021.12, `mpigxx`), which is the MPI the two parallel builds pick up. CUDA must be requested explicitly: the site default `cuda/11.4` is too old for the fetched Kokkos, and the builds here used `cuda/12.8` (`nvcc` at `/opt/apps/cuda/12.8/bin/nvcc`).

11.2 Configure

Four configurations are used, differing only in backend. All are configured from the repository root:

```

IGN=-DCMAKE_IGNORE_PATH=$PWD/cmake
COMMON="-D pgen=dpdm -D deposit=esirkepov -D shape_order=5 -D precision=double"
KOK="-D Kokkos_ENABLE_OPENMP=ON -D Kokkos_ENABLE_SERIAL=OFF"

# 1. CPU, OpenMP          -- validation, fast resonance case
cmake -B build-dpdm        $COMMON $KOK -D Kokkos_ARCH_ZEN3=ON $IGN
# 2. CPU + MPI            -- multi-node fallback
cmake -B build-dpdm-mpi    $COMMON $KOK -D Kokkos_ARCH_ZEN3=ON -D mpi=ON $IGN
# 3. CUDA, single GPU     -- the production path
cmake -B build-dpdm-cuda   $COMMON -D Kokkos_ENABLE_CUDA=ON \
    -D Kokkos_ARCH_AMPERE80=ON $IGN
# 4. CUDA + MPI, multi-GPU -- LZ 0.1, long-term stability
cmake -B build-dpdm-cuda-mpi $COMMON $KOK -D mpi=ON -D gpu_aware_mpi=OFF \
    -D Kokkos_ENABLE_CUDA=ON -D Kokkos_ARCH_AMPERE80=ON $IGN

```

Kokkos_ARCH.ZEN3 targets the AMD EPYC 7763 CPU nodes and Kokkos_ARCH.AMPERE80 the A100s; configuration 3 omits \$KOK because a CUDA backend does not need a threaded host backend for this workload. Each `cmake -B` re-fetches and re-builds its own private copy of Kokkos and ADIOS2 under `<build>/_deps`, which is why four directories cost ~ 1.2 GB rather than four small binaries.

Historical note. Configuration 1 was in fact first configured with `-D adios2_DIR=$PWD/build/_deps/adios2-build` reusing the ADIOS2 that an earlier unrelated `build/` directory had already fetched. That works, but it is not reproducible from a clean checkout — there is no `build/` to point at — so the form given above, using `$IGN` like the other three, is the one to follow. The two are equivalent: both end up at ADIOS2 2.11.0.

11.3 Compile

```

cmake --build build-dpdm        -j 8
cmake --build build-dpdm-mpi    -j 8
cmake --build build-dpdm-cuda   -j 8
cmake --build build-dpdm-cuda-mpi -j 8

```

Each produces its executable at `<build>/src/entity.xc`; that path is what the job scripts in `pgens/dpdm/` invoke. The CUDA configurations take substantially longer than the CPU ones — every kernel is compiled twice, to PTX and to Ampere SASS. Keep `-j` modest: LS6 login nodes are shared, and a large `-j` on an `nvcc` build is the usual way to get a job killed by the login-node watchdog.

Verify before trusting a build. Configure-time options fail silently here (Trap 2 below), so check the cache rather than assuming:

```

grep -E " ^Kokkos_ENABLE_(OPENMP|SERIAL|CUDA):BOOL" build-*/CMakeCache.txt

```

Trap 1: the ADIOS2 shim shadows its own package. `find_package(adios2 QUIET)` matches the repository’s own `cmake/adios2Config.cmake`, which only sets cache variables and exports neither a version nor any targets. CMake nevertheless sets `adios2_FOUND` and the fetch branch never runs. The symptom is ADIOS2: `v` with a path pointing into `entity/cmake`, followed by Target `"entity.xc"` links to `adios2::cxx_mpi` but the target was not found. Excluding that directory via `CMAKE_IGNORE_PATH` restores the intended fetch. This is an upstream bug: the shim shadows the package it exists to configure.

Trap 2: CMAKE_IGNORE_PATH also disables Kokkos OpenMP. The same exclusion blocks the repository’s Kokkos settings file, and a freshly fetched Kokkos defaults to **Serial only**. A Serial build is externally indistinguishable from a threaded one and runs $\sim 112\times$ slower (NLWP = 2, 99.5% CPU). Neither SLURM’s `-c` default nor `ibrun` binding produces this symptom, so there is no point looking there; the cache check at the end of the previous subsection is mandatory rather than advisory.

Trap 3: ADIOS2 target rename. ADIOS2 2.11 renamed the C++ bindings target from `cxx11` to `cxx` (and `cxx11_mpi` to `cxx_mpi`). ENTITY hardcodes the 2.11 names, so TACC’s `adios2/2.10.2` module cannot link even though it does provide `ADIOS2_HAVE_MPI`. Fetching 2.11.0 (ENTITY’s pinned tag) is the clean route, and requires a login node — compute nodes have no outbound network.

GPU-aware MPI. Disabled (`gpu_aware_mpi=OFF`). It would require `mvapich2-gdr-cuda12`, a different MPI stack from the `impi` in the `gcc13` hierarchy. The halo here is a few ghost cells of three field components, so host staging is free at this problem size.

12 Measured performance

This section records the throughput measurements that the sizing procedure of §7 draws on.

12.1 Throughput

Configuration	ms/step at $N_{\text{pc}} = 2 \times 10^5$	Note
1 CPU node (128 cores)	~ 950	steady state; rises +48% from 640
8 CPU nodes, MPI	135	$4.94\times$ speedup, 69% efficiency
1× A100	$265 \rightarrow 516$	$3.6\times$ a full CPU node
3× A100 (one node)	128	$2.07\times$ over 1 GPU, 69% efficiency

Table 2: Measured on 4×10^8 particles, $N_x = 1000$.

A single A100 outperforms an entire 128-core node by $3.6\times$. The feared collapse from deposition atomics on a 1000-cell grid did not occur: going from $N_{\text{pc}} = 2 \times 10^4$ to 2×10^5 — a tenfold increase in particles per cell, and therefore in atomic contention per cell — cost only a factor 1.42 ($0.444 \rightarrow 0.630$ ns per particle-step).

12.2 Per-step cost doubles after saturation

This governs all walltime planning.

On a single A100 at $N_{\text{pc}} = 2 \times 10^5$ the cost is 265 ms/step while the plasma is laminar and 516 ms/step once it saturates and becomes turbulent — particles disorder, and the memory locality the early rate depended on is lost. A 2000-step benchmark ($\omega_p t = 40$) shows a dead-flat 265 ms, which is *not* evidence of stability: the run saturates at $\omega_p t \approx 2000$, so such a benchmark never leaves the laminar phase. Taken at face value it overran a 48 h wall at 88% completion on the slow resonance case, which then required a checkpoint restart.

Rule: benchmark across the physics transition, or budget a factor two.

Part III

Runs, settings, and comparison with the paper

13 Parameters taken from the paper source

All values below are quoted from `main.tex` in the arXiv source package, not inferred from the text or figures.

Quantity	Paper value	This work
Grid	$N_x = 1000, L = 40 c/\omega_p$	identical
Cell size	$\Delta x = 0.04 c/\omega_p$	identical
Debye length	$\lambda_D \approx 0.033 c/\omega_p$	identical
Temperatures	$k_B T_e = k_B T_i = 10^{-3} m_e c^2$	identical
Mass ratio	$m_i/m_e = 1836$	identical
Thermal speed	$v_{th,e}^2 \equiv \langle (v_x - \langle v_x \rangle)^2 \rangle = k_B T/m$	identical (1V)
Thermal energy density	$5 \times 10^{-4} n_e m_e c^2$	4.9963×10^{-4} measured
Deposition	5th-order splines	<code>esirkepov, shape_order=5</code>
Pusher	Vay (2008)	Vay
Timestep	$\Delta t \approx 0.02/\omega_p$	CFL = 0.5 $\Rightarrow$ 0.02
Initial loading	equally spaced, co-located	<code>§10</code>
Shot noise	$\epsilon_{noise} = L^2/(12N_p N_x)$	agrees to $3.1 \times$
Resonance N_{pc}	2×10^2 and 2×10^5	identical
LZ sweep	$\omega(t) = 0.8\omega_p + 0.1\omega_p t/5000$	identical
LZ amplitudes	$v_q^D/v_{th}^e = 1.0$ and 0.1	identical
LZ N_{pc} criterion	$\bar{t}_{noise} \leq 0.02$	$2.7 \times 10^3, 2.7 \times 10^5$

Table 3: Paper parameters and the corresponding settings used here.

Two points deserve emphasis.

The 1V definition. The paper defines the thermal speed from the *one-dimensional* x -component variance, $v_{th,e}^2 = \langle (v_x - \langle v_x \rangle)^2 \rangle = k_B T/m$. ENTITY’s Maxwellian is isotropic 3V. Because there are no transverse forces in a 1D electrostatic run, the perpendicular degrees of freedom merely carry constant energy; including them would dilute T_e by a factor three. The quiet loader therefore sets $u_{x2} = u_{x3} = 0$ (`one_v = true`), matching SHARP exactly and removing the 3V-versus-1V correction from every downstream comparison. The paper’s quoted thermal energy density 5×10^{-4} matches the 1V value and is used as the reference line throughout.

The Landau–Zener crossing time. With $\omega_0 = 0.8$ and $\dot{\omega} = 2 \times 10^{-5}$, resonance ($\omega = \omega_p$) is crossed at $\omega_p t = 10^4$. The paper’s statement that “nonlinearity kills the resonant transfer around $\omega_p t = 5000$ ” refers to $\omega = 0.9\omega_p$, i.e. *before* the crossing — off-resonant excitation, not the crossing itself.

14 Preparatory and calibration runs

Before the production runs, a sequence of measurements pinned the unit conventions and validated the drive. These are summarised here because several production choices depend on them.

- **Units.** `skindepth0 = 1` with `densities = [2.0]` gives $n_e = n_0$ exactly, hence $\omega_{pe} = 1$. Confirmed by `T00.1` rest-mass part = 1.0000.
- **Energy conservation.** Over 2×10^4 steps, $\Delta T^{00} = -\frac{1}{2}\Delta\langle E^2 \rangle$ with $r = -0.996$; no grid heating.

- **ω_p calibration.** A five-point detuning scan ($\omega = 0.96\text{--}1.04$) is symmetric about $\omega = 1$ and follows the linear-theory sinc^2 envelope; the on-resonance case grows as t^2 across two decades in energy while every detuned case remains bounded.
- **Drive normalisation.** Least squares over $10 < \omega_p t < 40$ gives $C = 0.12432$ against Eq. A25’s $1/8 = 0.125$ (0.5%). The ion response ratio is 5.70×10^{-4} against $1/1836 = 5.45 \times 10^{-4}$, confirming the drive reaches both species with the correct q/m .

A factor-of-two trap. It is the *sum* $\epsilon_E + \Delta K E_e$ that obeys the t^2 law, not the field energy alone. The resonantly driven Langmuir wave equipartitions, so the field carries only half the wave energy on cycle average and swings between 3% and 85% of it instantaneously at $2\omega_p$. The paper’s linear-theory line must be compared against the sum.

15 The two resonance cases (paper Fig. 2)

Both run to $\omega_p t = 10^4$ with quiet loading, 1V, on a single A100.

Case	N_{pc}	A_0	C	peak $\epsilon_E/\epsilon_{\text{th}}$	final $\Delta K E_e/\epsilon_{\text{th}}$
fast, $v_q^D/v_{\text{th}}^e = 3 \times 10^{-2}$	2×10^2	9.48683×10^{-4}	0.1264	43.6 (at $\omega_p t = 553$)	69.6
slow, $v_q^D/v_{\text{th}}^e = 1 \times 10^{-3}$	2×10^5	3.16228×10^{-5}	0.1237	2.1 (at $\omega_p t = 3694$)	2.4

Table 4: Production resonance runs. C is the measured coefficient of $\epsilon_E + \Delta K E_e = C A_0^2 (\omega_p t)^2$; linear theory gives $1/8 = 0.125$. $\epsilon_{\text{th}} = 5 \times 10^{-4}$ is the 1V electron thermal energy density.

Initial temperature. $k_B T_e(0) = 4.9963 \times 10^{-4}$ against the 1V target 5.0×10^{-4} . The -0.075% residual is *not* loading error but the relativistic definition $T^{00} = \sum w m \gamma$, whose next term is $-\frac{3}{8}\langle u^4 \rangle = -0.075\%$. The ions, 1836 times heavier and far more non-relativistic, land at 4.999998×10^{-4} , which confirms the interpretation.

The regime distinction. The paper separates slow and fast growth at $v_q^D/v_{\text{th}}^e \sim (m_e/m_p)^{1/2}/2 \approx 0.0117$. Both of our cases overshoot the thermal energy in the *field* energy (43.6 versus 2.1), but the lasting electron heating differs by a factor ~ 29 : $O(10^2)$ for the fast case, versus a slow case whose electrons are barely changed — which is the paper’s own phrasing for the Landau–Zener result (“the kinetic energy of the electrons is not even changed by a factor of 2”). **The discriminator is the final electron heating, not the peak field energy.**

Saturation is delayed relative to random-start runs. With the noise floor eleven orders of magnitude lower at $t = 0$, daughter waves grow from a far smaller seed, so saturation occurs later. The fast case peaks at $\omega_p t = 553$ rather than the ~ 110 ($t_{\text{sat}} + 1/\omega_{pi}$) seen in earlier random-start scans. This is expected behaviour of a correct quiet start, and it means random-start timings are not a valid comparison.

Cost. The slow case required ~ 51 GPU-hours in total: 48 h to reach 88% completion, then a 14.5 h checkpoint restart — a direct consequence of the post-saturation slowdown of §12.2.

16 Landau–Zener runs

The Landau–Zener family is the physically realistic case: in cosmology it is $\omega_p(t)$ that drifts through the dark-photon mass, and the duration of the real transition is $\sim 10^{10}$ plasma periods, far beyond any simulation. The paper instead sweeps $m_{A'}(t)$ at a numerically tractable rate corresponding to $\omega_p/H \approx 7 \times 10^4$.

Run	v_q^D/v_{th}^e	N_{pc}	Resources	Status
Heating gate (undriven)	—	2.7×10^3	1 A100, 2 h	complete: T_e drift -0.016%
LZ high amplitude	1.0	2.7×10^3	1 A100, ~ 2 h	running
LZ low amplitude	0.1	2.7×10^5	3 A100, ~ 81 h chained	written, gated

Particle counts. The Landau–Zener noise criterion differs fundamentally from the resonant one. Before resonance the drive is a *bounded* forced oscillation of amplitude $A_0/(1 - \omega^2/\omega_p^2) = 2.78A_0$ at $\omega = 0.8\omega_p$, not a growing one, so there is no time at which the signal outgrows the noise. The paper requires $\bar{t}_{\text{noise}} = L/(A_0\sqrt{3N_p N_x/2}) \leq 0.02$ — driven energy fifty times the shot noise — which gives $N_{\text{pc}} = 2.7 \times 10^3$ and 2.7×10^5 for the two amplitudes. Note that the quiet start does *not* relax this requirement (§10.4).

Why the low-amplitude case needs three GPUs. At $N_{\text{pc}} = 2.7 \times 10^5$ the run holds 5.3×10^8 particles (≈ 31 GB), which is tight on a single 40 GB A100 and comfortable at 10.5 GB per GPU across three. Combined with the post-saturation slowdown, a single GPU would need ~ 168 h; three reduce this to ~ 81 h. Because the checkpoint domain count is locked, this must be chosen at first submission.

17 Superseded results

Three figures appearing in earlier drafts of these notes, and in comments still present in some older scripts, are wrong. They are listed so that they are recognisable if encountered — not as a history.

Superseded claim	Correct statement
“The shot-noise floor is $650\times$ below Eq. B4.”	A units error. N_p in $\epsilon_{\text{noise}} = L^2/(12N_p N_x)$ is the <i>total</i> macroparticle count, which the paper states explicitly; it was read as per-cell. Correctly evaluated, Eq. B4 agrees with measurement at a flat $3.1\times$ across three decades of N_{pc} , and is the right tool for sizing runs (§7).
“A quiet start removes the large- N_{pc} requirement.”	It does not. The ordered state relaxes to the thermal shot-noise floor by $\omega_p t \approx 5000$ (§10.4); the paper uses a quiet start <i>and</i> still requires large N_{pc} . A quiet start buys a clean $t = 0$, nothing more.
“Per-step cost is stable.”	It roughly doubles once the plasma saturates (§12.2). A benchmark confined to the laminar phase — 0.4% of a production run, and entirely before the physics transition — measures the wrong number.

All three share a failure mode worth guarding against generally: each extrapolated a measurement taken outside the regime where it was then applied. The cost model in particular was calibrated at $N_{\text{pc}} \leq 8000$ and subsequently failed in both directions, predicting 8 minutes for a run that took 41, and 97 seconds of stepping for one that exceeded 40 minutes.

18 Status and remaining work

Complete. Both resonance cases of Fig. 2, the quiet-start implementation and its validation (including MPI decomposition independence), the numerical-heating gate at Landau–Zener duration, and the full performance characterisation.

In progress. Landau–Zener at $v_q^D/v_{\text{th}}^e = 1.0$; the 0.1 case is written and gated on it.

Open.

- **No comparison against the published figures.** Every number here is internally consistent and matches the paper’s *text*, but the figure PDFs in the source package have not been digitised. The peak $\epsilon_E/\epsilon_{\text{th}}$ and its timing are the sharpest available discriminators.
- **Long-term stability** ($\omega_p t \rightarrow 8 \times 10^4$), the paper’s strongest claim, is untested. At $N_{\text{pc}} = 2 \times 10^5$ it is roughly eight times the slow resonance case and would require multi-GPU execution with chaining.
- **2D cross-check** not run; 1+1D sufficiency is assumed on the paper’s word.
- **Collisions.** ENTITY, like SHARP, is collisionless. The paper treats the final ion-acoustic damping analytically via τ_{ei} . This matches the paper’s regime, but “persists indefinitely” should be read as “persists on collisionless timescales”.